

Rúbrica del equipo SGROAS

Datos del equipo

Proyecto	Gestión de flota de transporte
Repositorio	https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb
Integrantes	Escudero Plaza María del Rosario; Tejada Bajaña Luis Alejandro; Castro Espinoza Kevin Moisés
Punto de partida	Mi revisión del 2 de septiembre de 2026, rama <code>main</code> , commit <code>538693d</code>
Qué se evalúa	Los 20 puntos que les pedí para el examen final (semana 19, del 7 al 11 de septiembre de 2026)

Reglas del juego

Esta rúbrica es del equipo y la nota que sale de aquí es la misma para el equipo. No reparto tareas: quién hace qué lo deciden ustedes. Yo reviso el resultado, punto por punto, en el repositorio.

Criterios de piso — si falla uno la nota es CERO

Antes de mirar cualquier punto compruebo estas tres condiciones. Si una falla, la nota es cero y no reviso nada más.

Piso 1. El PDF del SGA. Suben al SGA un PDF de una sola página: la carátula con los datos de identificación (universidad, carrera, asignatura, nombre del equipo, integrantes, docente y fecha) y la URL del repositorio escrita completa en una sola línea. Si la URL está rota, si el repositorio pide iniciar sesión, si la URL está partida en dos líneas o si el PDF tiene más de una página, la nota es cero.

Piso 2. El informe se regenera. Clono el repositorio, sigo el `README.md` al pie de la letra y compilo el `.tex`. El `README` tiene que decir qué compilador usar, en qué orden van los comandos, cuál es el archivo principal y qué dependencias hacen falta. Si el PDF no sale, o si el `README` no lo explica, la nota es cero.

Piso 3. Autoría limpia. Nadie hace commits a nombre de otro, no hay correos inventados y el repositorio es de ustedes. Si no, la nota es cero.

Qué commit reviso

Reviso el commit al que apunte el tag `v1.0.0` a la hora de cierre del examen final fijada en el SGA. Lo que no esté dentro de ese tag, para mí no existe. **Muevan el tag al último commit que quieren que revise:** si lo dejan donde está hoy, reviso el commit viejo y todo lo que hicieron después no cuenta. Si no hay tag, reviso el último commit de la rama principal.

Tres reglas más que aplican a todos los puntos

1. Una parte cuenta solo si está completa. “Casi lo terminamos” no suma.
2. Si en lo que corrigen o agregan aparece un dato, una cifra, una salida de herramienta o

una referencia inventada, el punto donde aparece queda en Nada (y con eso la nota queda en cero, por cómo funciona el cálculo).

3. Lo que tengan que explicarme en persona se conversa con el equipo completo, antes del cierre.

Cómo se calcula la nota

Cada punto tiene un peso, y los pesos de todos los puntos suman 10. En cada punto les pongo uno de cinco niveles: Completo, Casi, A medias, Apenas o Nada. El nivel les da dos cosas a la vez: **puntos** (Completo es todo el peso, Casi las tres cuartas partes, A medias la mitad, Apenas la cuarta parte y Nada cero) y un **factor** que multiplica. La nota es la multiplicación de todos los factores por la suma de todos los puntos.

$$\text{Nota} = (f_1 \times f_2 \times \cdots \times f_{20}) \times (p_1 + p_2 + \cdots + p_{20})$$

En la práctica esto significa tres cosas. Primero, si un solo punto queda en Nada su factor es cero, y cero por cualquier cosa es cero: la nota completa es cero, aunque todo lo demás esté perfecto. Segundo, lo que queda a medias les cuesta dos veces: resta en la suma y además baja el multiplicador, y baja más cuanto más pesa el punto. Tercero, si hacen todo completo la nota es 10, porque todos los factores valen 1 y los puntos suman 10.

De dónde sale cada factor

Para cada punto, el factor es la fracción lograda elevada a su peso dividido para diez: $f_i = (p_i/w_i)^{w_i/10}$, donde w_i es el peso y p_i los puntos obtenidos. No tienen que calcularlo: ya está hecho en la tabla de abajo. Así, un punto que pesa el doble que otro baja el doble el multiplicador cuando queda a medias, y el tamaño del equipo o el número de puntos no cambia la dureza de la regla: si todo queda al mismo nivel, la nota es la misma para cualquier equipo.

Así se mueve la nota de ustedes

- Todo Completo: $1 \times 10 = \mathbf{10,00}$.
- Todo Completo menos el punto 1 (el que más pesa: 0,9) en Casi: $0,974 \times 9,775 = \mathbf{9,53}$.
- Todo Completo menos el punto 1 en A medias: $0,940 \times 9,55 = \mathbf{8,97}$.
- Todo en Casi: $0,75 \times 7,50 = \mathbf{5,63}$. Todo en A medias: $0,50 \times 5,00 = \mathbf{2,50}$.
- Todo Completo menos el punto 5 (uno de los que menos pesa: 0,2) en Nada: $\mathbf{0,00}$. Sí: cero.

Tabla de pesos y factores

Cada celda trae dos números: arriba los puntos que suma ese nivel y abajo el factor que multiplica. Una raya significa que ese nivel no existe para ese punto (por ejemplo, en los puntos de sí o no).

#	Qué hay que hacer	Peso	Completo	Casi	A medias	Apenas	Nada
1	DOI inexistentes y el documento de verificación	0,9	0,900 ×1,000	0,675 ×0,974	0,450 ×0,940	0,225 ×0,883	0 ×0
2	SpotBugs ejecutado de verdad	0,5	0,500 ×1,000	—	—	—	0 ×0
3	ZAP: ejecutarlo o retirar la afirmación	0,5	0,500 ×1,000	—	0,250 ×0,966	—	0 ×0
4	@EnableMethodSecurity	0,4	0,400 ×1,000	—	—	—	0 ×0
5	Kevin Castro en la portada	0,2	0,200 ×1,000	—	—	—	0 ×0
6	El error 500 de asignaciones	0,5	0,500 ×1,000	—	0,250 ×0,966	—	0 ×0
7	k6 completo	0,5	0,500 ×1,000	—	0,250 ×0,966	0,125 ×0,933	0 ×0
8	Lighthouse completo	0,4	0,400 ×1,000	—	0,200 ×0,973	—	0 ×0
9	Informe que compila y README ejecutable	0,5	0,500 ×1,000	—	0,250 ×0,966	—	0 ×0
10	make all desde un clon limpio	0,4	0,400 ×1,000	—	0,200 ×0,973	—	0 ×0
11	Validador de trazabilidad honesto	0,4	0,400 ×1,000	—	0,200 ×0,973	—	0 ×0
12	Cobertura de ramas	0,5	0,500 ×1,000	—	0,250 ×0,966	—	0 ×0
13	SUS con quince y demografía correcta	0,6	0,600 ×1,000	—	0,300 ×0,959	—	0 ×0
14	Resúmenes y figuras en el PDF	0,5	0,500 ×1,000	—	0,250 ×0,966	0,125 ×0,933	0 ×0
15	Zenodo y checksums	0,3	0,300 ×1,000	—	0,150 ×0,979	—	0 ×0
16	Cookie segura y token fuera del JSON	0,4	0,400 ×1,000	—	0,200 ×0,973	—	0 ×0
17	Listados con código que existe	0,4	0,400 ×1,000	—	0,200 ×0,973	0,100 ×0,946	0 ×0
E1	Nombres del código en inglés	0,8	0,800 ×1,000	0,600 ×0,977	0,400 ×0,946	0,200 ×0,895	0 ×0
E2	Comentarios que generan documentación (Javadoc)	0,9	0,900 ×1,000	0,675 ×0,974	0,450 ×0,940	0,225 ×0,883	0 ×0
E3	Figuras en inglés	0,4	0,400 ×1,000	0,300 ×0,989	0,200 ×0,973	0,100 ×0,946	0 ×0
Suma de pesos		10,0	En cada celda: arriba los puntos, abajo el factor multiplicador.				

Punto por punto

Punto 1 — DOI inexistentes y el documento de verificación

Peso: 0,9 de 10

Origen: punto 1 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Corrijan los tres DOI que no existen (los correctos se los indiqué en el criterio D6) y retiren VERIFICACION-REFERENCIAS.md o rehágalo de verdad. Un documento que certifica que no hay referencias fabricadas y que se desmiente en diez segundos es lo que más daña su credibilidad.

Cómo lo reviso. Resuelvo cada DOI y compruebo que corresponda a la obra citada; leo el documento de verificación si lo mantienen.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) primer DOI corregido.
- (b) segundo DOI corregido.
- (c) tercer DOI corregido.
- (d) VERIFICACION-REFERENCIAS.md retirado o rehecho y cierto.

Completo Las 4 partes.

Casi 3 de 4 partes.

A medias 2 de 4 partes.

Apenas 1 de 4 partes.

Nada Ninguna parte.

Punto 2 — SpotBugs ejecutado de verdad

Peso: 0,5 de 10

Origen: punto 2 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Sustituyan el `spotbugsXml.xml` por la salida real de la herramienta. El plugin ya está bien configurado en su pom: solo hay que ejecutarlo.

Cómo lo reviso. Abro el XML y compruebo que tenga la estructura, la versión y la marca de tiempo que deja SpotBugs, y que sus clases existan en el código.

Cómo lo califico.

Completo Está hecho tal como se pide.

Nada No está hecho, o está a medias. Aquí no hay término medio.

Punto 3 — ZAP: ejecutarlo o retirar la afirmación

Peso: 0,5 de 10

Origen: punto 3 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Ejecuten el escaneo ZAP, o retiren del resumen la afirmación de haberlo hecho. Hoy el abstract afirma una evidencia que su propio directorio declara pendiente.

Cómo lo reviso. Busco el reporte de ZAP versionado y la afirmación en todo el entregable.

Cómo lo califico.

Completo Hay reporte real de ZAP y lo reportan, o la afirmación desapareció de todo el entregable.

A medias La quitaron de unos lugares y de otros no.

Nada Todo sigue igual.

Punto 4 — @EnableMethodSecurity**Peso:** 0,4 de 10

Origen: punto 4 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Añadan `@EnableMethodSecurity`. Sin ella, las ocho `@PreAuthorize` de `ReporteController` no hacen absolutamente nada.

Cómo lo reviso. Busco la anotación y pruebo un endpoint de reportes con un usuario sin permiso: tiene que devolver 403.

Cómo lo califico.**Completo** Está hecho tal como se pide.**Nada** No está hecho, o está a medias. Aquí no hay término medio.**Punto 5 — Kevin Castro en la portada****Peso:** 0,2 de 10

Origen: punto 5 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Pongan a Kevin Castro en la portada del informe. Tiene 53 commits y no puede faltar.

Cómo lo reviso. Abro la portada del PDF.

Cómo lo califico.**Completo** Está hecho tal como se pide.**Nada** No está hecho, o está a medias. Aquí no hay término medio.**Punto 6 — El error 500 de asignaciones****Peso:** 0,5 de 10

Origen: punto 6 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Arreglen el 500 de `GET /api/asignaciones` en producción y bajen de “verificado” los requisitos que dependen de él mientras no funcione.

Cómo lo reviso. Llamo al endpoint en el despliegue, con sesión iniciada.

Cómo lo califico.**Completo** Responde 200 con datos en producción.**A medias** Sigue en 500, pero los requisitos que dependen de él ya no dicen “verificado”.**Nada** Todo sigue igual.**Punto 7 — k6 completo****Peso:** 0,5 de 10

Origen: punto 7 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Suban a cinco corridas de k6, añadan el escenario de caché fría y ejecuten su propio `nonparametric.py` sobre los datos. Tienen la herramienta escrita y sin usar.

Cómo lo reviso. Cuento corridas crudas por escenario y busco la salida del script y su uso en el documento.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) cinco corridas crudas.
- (b) escenario de caché fría.
- (c) `nonparametric.py` ejecutado y reportado.

Completo Las 3 partes.**A medias** 2 de 3 partes.

Apenas 1 de 3 partes.
Nada Ninguna parte.

Punto 8 — Lighthouse completo

Peso: 0,4 de 10 Origen: punto 8 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Completen Lighthouse: tres corridas por perfil, incluido escritorio, contra su despliegue público, que ya funciona.

Cómo lo reviso. Abro los JSON y miro el perfil y la URL de cada corrida.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) tres corridas móvil contra el despliegue.
- (b) tres corridas escritorio contra el despliegue.

Completo Las 2 partes.
A medias 1 de 2 partes.
Nada Ninguna parte.

Punto 9 — Informe que compila y README ejecutable

Peso: 0,5 de 10 Origen: punto 9 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Corrijan el envoltorio `docs/informe-final.tex`, que no compila, y pongan en el README instrucciones que se puedan ejecutar tal cual, con el directorio y el número de pasadas. Ojo: esto también es el piso 2.

Cómo lo reviso. Clono y sigo el README al pie de la letra.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) `docs/informe-final.tex` compila.
- (b) README con directorio, comandos y número de pasadas.

Completo Las 2 partes.
A medias 1 de 2 partes.
Nada Ninguna parte.

Punto 10 — make all desde un clon limpio

Peso: 0,4 de 10 Origen: punto 10 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Pongan el bit de ejecución a `mvnw` y a los scripts, para que `make all` no muera con Permission denied desde un clon limpio, y añadan el objetivo `pdf`.

Cómo lo reviso. Ejecuto `make all` y `make pdf` desde un clon limpio.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) `make all` corre sin Permission denied.
- (b) objetivo `pdf` que funciona.

Completo Las 2 partes.
A medias 1 de 2 partes.
Nada Ninguna parte.

Punto 11 — Validador de trazabilidad honesto**Peso:** 0,4 de 10

Origen: punto 11 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Arreglen el validador de trazabilidad, que emite cuatro errores y aun así sale con código cero, y corrijan las seis filas que citan `ReportesServiceTest`, que no existe.**Cómo lo reviso.** Ejecuto el validador: si hay errores, tiene que salir con código distinto de cero. Compruebo cada prueba citada.**Cómo lo califico.** Cuento estas partes; una parte cuenta solo si está completa:

- (a) el validador sale con error cuando hay errores.
- (b) las seis filas citan pruebas que existen.

Completo Las 2 partes.**A medias** 1 de 2 partes.**Nada** Ninguna parte.**Punto 12 — Cobertura de ramas****Peso:** 0,5 de 10

Origen: punto 12 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Suban la cobertura de ramas: 18,18 % en controladores y 51,22 % en servicios. La meta es la de la guía: 70 % o más.**Cómo lo reviso.** Recalculo desde el `jacoco.xml` versionado.**Cómo lo califico.** Cuento estas partes; una parte cuenta solo si está completa:

- (a) controladores: ramas 70 % o más.
- (b) servicios: ramas 70 % o más.

Completo Las 2 partes.**A medias** 1 de 2 partes.**Nada** Ninguna parte.**Punto 13 — SUS con quince y demografía correcta****Peso:** 0,6 de 10

Origen: punto 13 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Suban el SUS a quince participantes reales y corrijan la demografía del capítulo 5, que sus propios datos desmienten.**Cómo lo reviso.** Cuento respuestas en el CSV crudo, recalculo con Brooke y comparo la demografía con los datos. Datos que no se sostienen dejan este punto en Nada.**Cómo lo califico.** Cuento estas partes; una parte cuenta solo si está completa:

- (a) quince participantes o más.
- (b) demografía del capítulo 5 igual a los datos.

Completo Las 2 partes.**A medias** 1 de 2 partes.**Nada** Ninguna parte.**Punto 14 — Resúmenes y figuras en el PDF****Peso:** 0,5 de 10

Origen: punto 14 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Suban los resúmenes a 200–250 palabras (hoy tienen 193 y 169) y metan los diagramas C4 y las figuras en el PDF: hoy los tres labels de figura están huérfanos.

Cómo lo reviso. Cuento las palabras del resumen y del abstract en el PDF del informe (sin título ni palabras clave). Busco las figuras en el PDF, referenciadas en el texto.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) resumen entre 200 y 250.
- (b) abstract entre 200 y 250.
- (c) C4 y figuras en el PDF, referenciados.

Completo Las 3 partes.

A medias 2 de 3 partes.

Apenas 1 de 3 partes.

Nada Ninguna parte.

Punto 15 — Zenodo y checksums

Peso: 0,3 de 10

Origen: punto 15 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Actualicen el depósito de Zenodo del software a v1.0.0 con la licencia correcta, y regeneren el manifiesto de checksums con finales de línea consistentes.

Cómo lo reviso. Resuelvo el DOI y verifico el manifiesto con `sha256sum -c`.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) Zenodo en v1.0.0 con la licencia correcta.
- (b) manifiesto que verifica sin errores.

Completo Las 2 partes.

A medias 1 de 2 partes.

Nada Ninguna parte.

Punto 16 — Cookie segura y token fuera del JSON

Peso: 0,4 de 10

Origen: punto 16 de mi revisión del 2 de septiembre.

Qué tienen que hacer. Pongan `Secure` a la cookie en producción y dejen de devolver el token en el cuerpo JSON.

Cómo lo reviso. Inicio sesión en el despliegue y miro las cabeceras y el cuerpo de la respuesta.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) cookie con `Secure`.
- (b) sin token en el cuerpo JSON.

Completo Las 2 partes.

A medias 1 de 2 partes.

Nada Ninguna parte.

Punto 17 — Listados con código que existe

Peso: 0,4 de 10

Origen: punto 17 de mi revisión del 2 de septiembre.

Qué tienen que hacer. El Listado 4 muestra un método que no existe, el Listado 3 un tipo que no existe y el Listado 1 dos elementos inexistentes. El documento tiene que mostrar el código que

hay.

Cómo lo reviso. Comparo cada listado con el código del árbol evaluado.

Cómo lo califico. Cuento estas partes; una parte cuenta solo si está completa:

- (a) Listado 1.
- (b) Listado 3.
- (c) Listado 4.

Completo	Las 3 partes.
A medias	2 de 3 partes.
Apenas	1 de 3 partes.
Nada	Ninguna parte.

Punto E1 — Nombres del código en inglés

Peso: 0,8 de 10

Origen: exigencia 1 de las tres que fijé para la Entrega Final.

Qué tienen que hacer. Los nombres dentro del código van en inglés: clases, interfaces, métodos, atributos y variables del backend. Los textos que el usuario ve en pantalla pueden seguir en español; lo que cuenta es cómo se llaman las cosas dentro del código.

Cómo lo reviso. Recorro los tipos y los métodos de `src/main/java` y cuento cuántos llevan una palabra en español. Saco dos porcentajes, uno de clases y otro de métodos, y me quedo con el peor de los dos. Hoy: nomenclatura mayoritariamente en español en clases y métodos.

Cómo lo califico.

Completo	5 % o menos con palabras en español.
Casi	Más de 5 % y hasta 15 %.
A medias	Más de 15 % y hasta 30 %.
Apenas	Más de 30 %, pero menos que lo que medí el 2 de septiembre.
Nada	Igual o peor que el 2 de septiembre.

Punto E2 — Comentarios que generan documentación (Javadoc)

Peso: 0,9 de 10

Origen: exigencia 2 de las tres que fijé para la Entrega Final.

Qué tienen que hacer. Cada método público del backend lleva su comentario Javadoc con `@param` por cada parámetro, `@return` si devuelve algo y `@throws` si lanza excepciones. Un `@param` que solo repite el nombre (por ejemplo, “*@param id el id*”) no cuenta: tiene que decir qué es y para qué sirve.

Cómo lo reviso. Cuento los métodos públicos de `src/main/java` y cuántos tienen el Javadoc completo según la regla de arriba. Para Completo, además, `mvn javadoc:javadoc` tiene que terminar bien. Hoy: cero `@param`, cero `@return` y cero `@throws` en 189 métodos públicos.

Cómo lo califico.

Completo	90 % o más de los métodos públicos, y <code>mvn javadoc:javadoc</code> termina bien.
Casi	Entre 70 % y menos de 90 % (o 90 % o más pero el comando falla).
A medias	Entre 40 % y menos de 70 %.
Apenas	Entre 10 % y menos de 40 %.
Nada	Menos de 10 %.

Punto E3 — Figuras en inglés**Peso:** 0,4 de 10

Origen: exigencia 3 de las tres que fijé para la Entrega Final.

Qué tienen que hacer. Todo el texto que va dentro de las figuras está en inglés: títulos, ejes, leyendas, etiquetas y las cajas de los diagramas (C4, entidad-relación, gráficos de resultados). Las capturas de la interfaz no cuentan, porque la pantalla puede estar en español.

Cómo lo reviso. Abro cada diagrama y cada gráfico del repositorio y del PDF, y cuento cuántos tienen todo su texto en inglés. Hoy: ninguna figura del repositorio tiene texto en inglés.

Cómo lo califico.**Completo** Todos.**Casi** 75 % o más, pero no todos.**A medias** Entre 50 % y menos de 75 %.**Apenas** Al menos uno, pero menos del 50 %.**Nada** Ninguno.

Hoja de cálculo

La lleno yo al revisar. Para cada punto anoto el nivel y copio de la tabla los puntos y el factor.

#	Punto	Peso	Nivel	Puntos	Factor
1	DOI inexistentes y el documento de verificación	0,9			
2	SpotBugs ejecutado de verdad	0,5			
3	ZAP: ejecutarlo o retirar la afirmación	0,5			
4	@EnableMethodSecurity	0,4			
5	Kevin Castro en la portada	0,2			
6	El error 500 de asignaciones	0,5			
7	k6 completo	0,5			
8	Lighthouse completo	0,4			
9	Informe que compila y README ejecutable	0,5			
10	make all desde un clon limpio	0,4			
11	Validador de trazabilidad honesto	0,4			
12	Cobertura de ramas	0,5			
13	SUS con quince y demografía correcta	0,6			
14	Resúmenes y figuras en el PDF	0,5			
15	Zenodo y checksums	0,3			
16	Cookie segura y token fuera del JSON	0,4			
17	Listados con código que existe	0,4			
E1	Nombres del código en inglés	0,8			
E2	Comentarios que generan documentación (Javadoc)	0,9			
E3	Figuras en inglés	0,4			
S = suma de puntos					
M = multiplicación de todos los factores					
Nota = $M \times S$ (dos decimales)					